SIC Transmetro — Lenguajes, motor de base de datos y métricas

Especificación técnica y plan de métricas de calidad · Proyecto final de Aseguramiento de la Calidad del Software · 2 de octubre de 2026

1. Lenguajes de programación

El sistema usa un solo lenguaje de programación, JavaScript, en el servidor y en el navegador, más SQL y PL/pgSQL en la base de datos. Eso reduce el costo de cambiar de contexto: quien mantiene una capa puede leer la otra.

Lenguaje	Estándar o versión	Dónde se usa	Por qué ese y no otro
JavaScript	ECMAScript 2022, módulos ES	API REST sobre Node.js 20 o superior	Mismo lenguaje que el frontend. Node tiene el mejor soporte para entrada y salida concurrente, que es lo que hace una API: esperar a la base y responder
JavaScript con JSX	ECMAScript 2022 + JSX	Aplicación web en React 18	JSX permite escribir la interfaz como componentes reutilizables. Se eligió JavaScript y no TypeScript para no sumar un paso de compilación a un equipo de una persona con fecha de entrega
SQL	SQL:2016, dialecto PostgreSQL	Definición del esquema, consultas y vistas	Es el lenguaje del motor. Se escribe SQL directo, sin ORM, para que las restricciones y los triggers queden a la vista

PL/pgSQL	**PostgreSQL 14+**	**15 funciones y 29 triggers de reglas de negocio**	**Lenguaje procedural nativo del motor. Permite que la regla se ejecute dentro de la misma transacción que el cambio**
HTML5 y CSS3	**Estándar vigente**	**Página contenedora y hoja de estilos propia**	**Sin framework de CSS: 369 líneas propias, responsivas, sin dependencias que mantener**
JSON	**RFC 8259**	**Formato de intercambio de la API y configuración**	**Estándar de facto para REST, y PostgreSQL lo almacena de forma nativa como jsonb en la bitácora**
Mermaid	**Versión 11**	**22 diagramas versionados como texto**	**Un diagrama en texto se versiona y se compara como código. Un diagrama en imagen, no**

Por qué JavaScript y no otro lenguaje de servidor

Alternativa	Por qué se descartó
C# con ASP.NET Core	**Es sólido y la opción natural si el equipo ya trabaja en .NET, pero obliga a dos lenguajes y dos cadenas de herramientas, y el frontend seguiría siendo JavaScript**
Java con Spring Boot	**Más ceremonia de configuración de la que justifica un sistema de este tamaño**
Python con Django o FastAPI	**Django impone su ORM y su propia idea de la capa de datos, que choca con poner las reglas en la base. FastAPI es buena opción, pero vuelve a sumar un segundo lenguaje**

PHP con Laravel	**Viable, pero su ecosistema empuja hacia la lógica en el modelo de la aplicación, no en el motor**

2. Motor de base de datos

PostgreSQL 14 o superior. El script se ejecutó y se verificó completo en PostgreSQL 18.3. Se despliega como servicio administrado en la nube.

Por qué PostgreSQL

El requisito del proyecto pedía un motor relacional de código abierto. Entre los candidatos reales, PostgreSQL es el único que trae las tres herramientas sobre las que descansa este diseño:

Característica	Para qué la usa el sistema	¿La tiene MySQL o MariaDB?
EXCLUDE USING gist sobre rangos de tiempo	Impedir que una unidad esté en dos líneas o dos parqueos en periodos que se traslapen (RF-04, RF-06)	No. Habría que emularlo con bloqueos en la aplicación, que falla con acceso concurrente
Columnas generadas y almacenadas	Calcular porcentaje de ocupación, sobrecupo y diferencia de conciliación (RF-09, RF-12)	Sí, desde 5.7, pero sin la misma cobertura de expresiones
Triggers con un lenguaje procedural completo	Abrir y cerrar alertas automáticas y escribir la bitácora (RF-05, RF-08, RF-16)	Limitados: un trigger por evento y por tabla hasta versiones recientes
Tipos **ENUM**, **jsonb**, **inet**, **citext**	Dominios cerrados, datos de auditoría, direcciones IP, correos sin distinguir mayúsculas	Parcial
Índices parciales y por expresión	Una sola asignación vigente por unidad, una sola alerta abierta por tipo y entidad	No

Otra alternativa	Por qué se descartó
SQL Server	No es de código abierto; su licencia en producción tiene costo
Oracle	Mismo problema de licencia, y mucho más motor del que este sistema necesita
SQLite	Sin concurrencia real de escritura ni usuarios; no sirve para varias estaciones escribiendo a la vez
MongoDB u otro documental	El dominio es fuertemente relacional: líneas, estaciones, recorridos, asignaciones. Sin integridad referencial, cada regla volvería a la aplicación

Cómo se usa el motor

Aspecto	Decisión
Esquema	Todo vive en el esquema `transmetro`, no en `public`, para separarlo de las extensiones y poder recrearlo completo
Extensiones	`btree_gist` (restricciones de exclusión), `citext` (usuario y correo), `pgcrypto` (funciones de hash)
Acceso	Un solo usuario de aplicación con permisos acotados. A la bitácora solo se le concede `INSERT` y `SELECT`
Conexión	Pool de hasta 10 conexiones, TLS obligatorio, `search_path` fijado a `transmetro, public`
Transacciones	Toda escritura va dentro de una transacción que publica el usuario con `SET LOCAL app.usuario_id`
Concurrencia	Bloqueo de fila con `SELECT ... FOR UPDATE` donde hay riesgo de carrera, más las restricciones `EXCLUDE` como última garantía
Migraciones	Scripts numerados en `01_database/`, versionados, ejecutables en orden con `npm run db:reset`

3. Métricas del producto

Medido el 2 de octubre de 2026 sobre el repositorio, excluyendo dependencias y archivos generados. El conteo de líneas incluye comentarios y líneas en blanco.

Tamaño del código

Lenguaje	Archivos	Líneas	Participación
JavaScript (backend y utilidades)	40	3,532	35 %
JavaScript con JSX (frontend)	18	2,278	23 %
SQL y PL/pgSQL	12	2,212	22 %
Mermaid (diagramas)	22	1,578	16 %
CSS	1	369	4 %
HTML y JSON	3	71	menos de 1 %
Total de código	96	10,040	100 %

A eso se suman 1,103 líneas de documentación en Markdown.

Tamaño del modelo de datos

Elemento	Cantidad
Tablas	32
Vistas	12
Tipos enumerados	31
Funciones PL/pgSQL	15
Triggers	29
Índices creados de forma explícita	37
Índices totales, incluidos los de llaves	104
Llaves foráneas	76
Restricciones CHECK	47
Restricciones UNIQUE	28

Restricciones EXCLUDE	**4**
Columnas generadas	**4**

Tamaño de la API y de la interfaz

Elemento	Cantidad
Endpoints REST	**51**
Módulos del backend	**9**
Pantallas del frontend	**11**
Roles con permisos diferenciados	**5**
Módulos protegidos por la matriz de permisos	**8**

4. Métricas de calidad

Estas son las métricas que se reportan en el informe del curso. Cada una indica cómo se calcula, con qué se mide, cuál es la meta y cada cuánto se revisa.

4.1 Métricas de producto

Métrica	Fórmula	Meta	Herramienta	Frecuencia
Cobertura de pruebas en servicios	Líneas ejecutadas por las pruebas entre líneas totales de src/modules	≥ 70 %	vitest --coverage	Al cierre de cada sprint
Cobertura de requerimientos	RF con al menos un caso de prueba automatizado entre RF totales	100 % de los RF con regla de negocio	Matriz de trazabilidad	Al cierre de cada sprint
Densidad de defectos	Defectos encontrados entre miles de líneas de código	Menos de 5 por cada mil	Registro de defectos	Al cierre de cada sprint
Defectos escapados	Defectos hallados después de la entrega entre defectos totales	Menos del 10 %	Registro de defectos	Al final del proyecto

Reglas garantizadas en la base	Reglas con restricción o trigger entre reglas de negocio totales	100 % de las reglas duras	Revisión del script	Al cierre de cada sprint
Tiempo de respuesta	Percentil 95 de los 10 endpoints más usados	Menos de 3 segundos	Pruebas de carga con k6	Sprint 7 y antes de entregar
Disponibilidad	Minutos en servicio entre minutos del horario operativo	≥ 99 %	Sonda /salud cada minuto	Mensual

4.2 Métricas de proceso

Métrica	Fórmula	Para qué sirve
Velocidad	Historias terminadas por sprint	Proyectar si el alcance cabe en el tiempo restante
Cumplimiento de la definición de terminado	Historias que cumplen los 7 criterios entre historias cerradas	Detectar deuda disfrazada de avance
Defectos por fase de detección	Defectos hallados en diseño, en pruebas y en producción	Es el argumento central del informe: un defecto hallado en diseño cuesta una fracción de lo que cuesta en producción
Retrabajo	Horas dedicadas a corregir entre horas totales	Medir el costo real de la mala calidad

Tiempo de corrección	**Promedio entre el reporte de un defecto y su cierre**	**Evaluar la capacidad de respuesta**

4.3 Estado actual de las métricas

Medido hoy, con el Sprint 0 cerrado.

Métrica	Valor actual	Meta	Estado
Pruebas automatizadas	**9, todas en verde**	**Crece con cada sprint**	**En curso**
Cobertura de pruebas en servicios	**Sin medir aún; las pruebas actuales cubren autenticación y permisos**	**≥ 70 %**	**Pendiente**
Reglas de negocio garantizadas en la base	**7 de 7 reglas duras: RF-04, RF-05, RF-06, RF-08, RF-09, RF-10, RF-16**	**100 %**	**Cumplido**
Reglas verificadas con datos inválidos	**9 pruebas ejecutadas contra PostgreSQL real, todas con el resultado esperado**	**100 % de las reglas duras**	**Cumplido**
Requerimientos implementados	**RF-01 a RF-06 y RF-15, es decir 7 de 16**	**16 de 16**	**En curso**
Trazabilidad requerimiento a implementación	**16 de 16 RF con su fila en la matriz**	**100 %**	**Cumplido**
Defectos encontrados y corregidos	**2: el conteo de alertas iniciales en los datos de ejemplo y una etiqueta encimada en un diagrama**	**Se registran todos**	**En curso**
Tiempo de respuesta	**Sin medir; falta la prueba de carga**	**Menos de 3 segundos**	**Pendiente**

4.4 Cómo se recolectan

```
npm test                    # pruebas automatizadas
npx vitest run --coverage  # cobertura de lineas y ramas
npm run db:reset           # recrea el esquema y verifica que el script
corre completo
```

La verificación del script contra un PostgreSQL real se hace con PGlite, que ejecuta el motor completo sin instalar un servidor. Eso permite correr las pruebas de reglas de negocio en cualquier equipo y, más adelante, dentro de integración continua.

5. Resumen

Pregunta	Respuesta
¿Qué lenguajes?	JavaScript ECMAScript 2022 en servidor y navegador, SQL y PL/pgSQL en la base, HTML5 y CSS3 en la interfaz
¿Qué motor de base de datos?	PostgreSQL 14 o superior, verificado en 18.3, administrado en la nube
¿Por qué ese motor?	Es el único motor libre con restricciones de exclusión sobre rangos, que son las que garantizan las reglas RF-04 y RF-06 sin depender de la aplicación
¿Cuánto se ha construido?	10,040 líneas de código, 32 tablas, 51 endpoints, 11 pantallas
¿Cómo se mide la calidad?	7 métricas de producto y 5 de proceso, con meta y frecuencia definidas